

# SuperReciteHelper 作业报告

——程设大作业第 16 小组

|      |                                                                                                             |
|------|-------------------------------------------------------------------------------------------------------------|
| 项目类型 | Python + PyQt6 桌面刷题工具                                                                                       |
| 项目仓库 | <a href="https://github.com/duyaOmega/SuperReciteHelper">https://github.com/duyaOmega/SuperReciteHelper</a> |
| 报告内容 | 程序功能、模块设计、小组分工、总结反思                                                                                         |

# 1. 程序功能介绍

SuperReciteHelper 是一个面向日常复习和考试准备的桌面刷题工具。项目的核心目标是把分散在 txt、docx 等文档中的题目整理为可反复练习的题库，并在练习过程中记录用户的作答情况，帮助用户发现薄弱题目并进行针对性复习。

当前版本的主线实现基于 Python 和 PyQt6。程序启动后，用户选择题库文件，系统完成题目解析并进入刷题界面。客观题可以直接选择选项并自动判分，主观题则先显示参考答案，再由用户自评答对或答错。所有作答记录会保存到本地，供后续抽题和统计使用。

## 1.1 核心功能

- 题库导入：支持 .txt, .doc, .docx 文件，适合课程复习资料、选择题题库和整理后的背诵材料。
- 题型识别：支持单选题、多选题、判断题、填空题和简答题，并根据选项结构、答案行和题干形式推断题型。
- 刷题练习：提供下一题、提交答案、显示结果、主观题自评等基本流程。
- 键盘作答：客观题可输入 A/B/C/D 或多选字母组合后回车提交，主观题可在显示答案后输入 t/f 自评。
- 错题加权：根据历史作答次数、错误次数、错误率和未做题状态调整抽题权重。
- 统计与改题：提供考频统计、当前题编辑、手动修改管理和重置记录等功能。
- 本地持久化：保存最近打开文件、做题记录和手动改题结果，关闭程序后仍可继续使用。
- DPI 自适应：程序启动时检测屏幕 DPI 并自动缩放界面元素，确保在不同分辨率显示器上都能正常显示。

## 1.2 基本使用流程

用户运行程序后选择题库文件，系统调用解析模块读取题目并生成题目列表。进入主界面后，用户点击“下一题”开始练习；客观题提交后立即显示正确或错误，主观题显示参考答案后由用户自行判断。作答完成后，记录会立即写入本地 JSON 文件，后续抽题会优先考虑错误率较高或尚未做过的题目。同时程序将对用户的作答情况进行统计。用户也可以手动修改题目或答案。

# 2. 项目各模块与类设计细节

项目采用按职责拆分的模块结构：入口、解析、题目编辑、记录持久化、抽题策略和图形界面相对独立。这种结构使解析逻辑和界面逻辑不会过度耦合，也便于后续替换界面框架或扩展题库格式。

| 模块      | 主要文件             | 设计要点                                          |
|---------|------------------|-----------------------------------------------|
| 启动与文件选择 | main.py          | 实例化 QApplication，选择题库文件，调用解析函数，并启动 PyQt6 主窗口。 |
| 题库解析    | parser.py        | 读取 txt/docx 纯文本，规范化文本，切分题块，识别选项、题型和答案。        |
| 题目编辑校验  | question.py      | 格式化答案，解析手动输入的选项和答案，并按题型校验输入合法性。               |
| 数据持久化   | question_bank.py | 管理应用状态、错题记录、手动改题记录，并用稳定哈希键标识题目。               |
| 抽题策略    | session.py       | 根据错误次数、错误率和未做题状态计算权                           |

|          |                    |                                   |
|----------|--------------------|-----------------------------------|
|          |                    | 重，完成加权随机抽题。                       |
| 连接前后端    | ui_main.py         | 控制刷题主界面、题库状态与交互变量                 |
| PyQt 主界面 | ui_pyqt.py         | 实现刷题主窗口、题目展示、作答交互、键盘输入、统计入口和编辑入口。 |
| PyQt 弹窗  | ui_pyqt_dialogs.py | 实现题目编辑、修改管理和考频统计等弹窗。              |

## 2.1 启动流程与主窗口设计

main.py 是当前 PyQt6 版本的入口。它负责读取上次打开文件路径作为初始目录，弹出文件选择框，将用户选择的题库文件传入 build\_parse\_candidates，并取默认“自动识别”方案中的题目列表。解析成功后，程序保存最近打开文件，再创建 QuizWindow 显示主界面。

QuizWindow 继承自 QMainWindow，内部维护当前题库、当前题目、作答记录、选项控件和提交状态。界面被划分为顶部信息栏、进度条区域、可滚动题目区域和底部操作栏。这样的布局使用户可以在同一个窗口中完成抽题、作答、查看反馈和进入工具弹窗。

在界面细节方面，QuizWindow 采用卡片式选项设计：每道题的选项被渲染为带圆角边框的 QFrame 卡片，内部嵌入 QRadioButton（单选/判断）或 QCheckBox（多选）以及选项文字标签。用户选中选项时卡片边框变为蓝色，提交后正确选项显示为绿色边框，错误选项显示为红色边框，未选中的选项则变为灰色，视觉反馈清晰直观。

界面上方设有 QProgressBar 进度条，实时显示已作答题目数占总数的比例，以及当前正确率百分比。程序首次启动时会显示欢迎界面，提示用户开始练习。此外，程序会通过 Qt 的 screen.devicePixelRatio() 和 logicalDotsPerInch() 计算 DPI 缩放因子，所有界面尺寸（字体、边距、卡片大小等）均乘以该因子，从而适配高分辨率屏幕。

## 2.2 题库解析模块

parser.py 是项目中逻辑最集中的模块。为了让用户使用更便捷、快速、不用付出额外代价，我们并没有选择直接接入 api，而是实现了一个较为通用的解析逻辑。解析过程大致分为文件读取、文本规范化、题块切分、单题解析、文末答案区回填和重新编号。

文件读取阶段需要处理多种编码。txt 文件会依次尝试 utf-8-sig、utf-8、utf-16-le、utf-16-be、gb18030 和 gbk 编码读取，以兼容不同来源的文本文件。docx 文件则通过 Python 标准库 zipfile 解压文档包，再用 xml.etree.ElementTree 解析 word/document.xml，逐段落提取 w:t 节点的文本内容，无需依赖第三方 docx 库。

文本规范化阶段会自动执行多项预处理：将全角字母转换为半角、把答案标签（如“正确答案：”“【答案】”）拆分到独立行、确保题号从新行开始等。这些处理使得后续的题块切分和答案提取更加稳定。

题块切分采用多层回退策略。首先尝试按题号正则表达式（匹配“1.”“1.”“一.”等格式）切分；若切分结果不足，则改为按答案行位置切分。单题解析阶段同时运行两套解析器——基于题块的 parse\_single\_block 和基于问答对的 \_parse\_numbered\_qa\_blocks，最终取解析结果更多的一组。

文末答案区支持两种常见格式：同行配对（如“12.A”“35.ABD”）和表格交替排列（题号行与答案行交错出现）。提取后的答案会按章节类型和题号匹配回填到对应题目中。后处理阶段还会将部分简答题自动转为填空题。

单题解析时，`parse_single_block` 会先拆出题干和答案，再识别 A-H 选项结构。若题目包含选项，则根据答案数量和判断题选项内容确定单选、多选或判断题；若没有选项，则根据题干中的空位符号判断填空题，否则归为简答题。对于文末统一答案区，`_extract_answer_keys_from_text` 会提取题号到答案的映射，再由 `_fill_answers_from_answer_keys` 回填到题目列表。

目前已开始尝试转向基于 DeepSeek API 的语义解析器，实现在 `llm_parser.py` 中。相较于基于正则匹配的解析逻辑，基于语义解析的灵活性大大提升，对输入源文件的格式要求大幅降低，进而可以提高匹配成功率并减少数据预处理的繁琐性。前端尚不支持该解析方式，日后会进一步扩展该实现。

## 2.3 题目编辑与数据持久化

由于自动解析不可能覆盖所有题库格式，项目提供手动改题能力。`question.py` 中的函数负责把答案统一格式化为可展示文本，并对用户在编辑弹窗中输入的答案和选项进行校验。例如，单选题和判断题必须只有一个有效选项，多选题可以有多个选项，填空题和简答题则要求答案文本不能为空。

`question_bank.py` 负责持久化。程序将数据保存到系统用户目录下的 `SuperReciteHelper` 文件夹中，主要包括 `app_state.json`、`error_record.json` 和 `question_edits.json`。为了避免题号变化导致记录错乱，每道题会根据原始题干、选项和题型生成 SHA1 稳定键，做题记录和手动修改都通过该键关联。

在数据结构层面，程序内部以字典表示每道题目，核心字段包括：`id`（序号）、`text`（题干）、`options`（选项字典，如 `{"A": "...", "B": "..."}`）、`answer`（客观题为选项字母列表，主观题为文本字符串）、`type`（题型代码：`single/multi/judge/blank/short`）以及 `source_no`（原始题号）。此外，每道题在加载时会自动生成 `_base_key` 和 `_record_key` 两个稳定标识字段。

SHA1 稳定键的生成依据是题型代码、规范化后的题干文本和排序后的选项内容，三者拼接后计算 SHA1 哈希值，格式为 `"q:{十六进制哈希}"`。这意味着即使同一题库被重新导入导致题号变化，只要题目内容不变，其稳定键就不变，做题记录和手动修改都能正确关联。`error_record.json` 以该键为索引存储每道题的作答次数和错误次数；`question_edits.json` 同样以该键为索引，存储修改后的题干、选项、答案和题型，同时保留原始值以便恢复。

## 2.4 抽题策略与统计反馈

`session.py` 中的 `weighted_random_pick` 是抽题策略核心。它为每道题计算权重：基础权重相同，错误次数越多、错误率越高，权重越大；从未做过的题也会获得额外权重，以减少用户一直刷不到新题的情况。最后程序在总权重区间内随机采样，选出下一道题。

具体的权重计算规则如下：基础权重为 1.0；错误次数达到 3 次加 1.0，达到 5 次加 2.5；错误率超过 30% 加 1.5，超过 50% 加 3.0；从未做过的题目额外加 2.0。此外，程序会动态追踪未做题的比例：当大部分题目已被做过、仅剩少量新题时，这些新题的权重会按 `"1 / 未做题比例"` 进一步放大，从而保证用户最终能刷到每一道题。

`session.py` 中还定义了 `PracticeSession` 类，封装了题库、记录和当前题目的状态，并提供 `pick_next()` 方法调用上述加权算法获取下一题。虽然当前 PyQt6 界面直接调用 `weighted_random_pick` 函数而非通过该类，但 `PracticeSession` 为未来扩展（如会话级统计、暂停恢复等）预留了接口。

统计功能主要由 `ui_pyqt_dialogs.py` 中的 `show_frequency_stats_dialog` 实现。它会汇总总题数、已作答题数、总作答次数、总错误次数和错误率，并支持按作答次数、错误次数、错误率或题号排序查看题目。用户还可以打开题目详情，从统计结果反查具体题干、选项和答案。

### 3. 小组成员分工情况

吴弈辰：项目灵感、架构与初版程序各模块。在后续更新中进行维护和修改，包括题库解析逻辑的更新（parser.py）、抽题加权逻辑（session.py）、DPI 缩放等。

纪涵予：将界面从 Tkinter 迁移至 PyQt6，重新设计 UI 布局，并将 ui\_main 拆分为主窗口与弹窗两个独立模块；清理 question.py 和 question\_bank.py 中与 Tkinter 相关的代码。

周浩钧：简化题库解析逻辑、消除 question.py 和 question\_bank.py 中的冗余实现；重组代码框架以便于维护和扩展；尝试向基于 LLM API 的解析思路转型。

### 4. 项目总结与反思

SuperReciteHelper 的开发围绕一个比较明确的实际需求展开：复习时只阅读资料很难确认掌握程度，因此需要一个能够抽题、判题、记录错误并反复练习的工具。从当前实现看，项目已经完成了从题库导入到刷题练习、记录保存、错题统计和手动修题的完整闭环，具备作为课程复习辅助工具的基本可用性。

#### 4.1 项目收获

- 模块拆分提高了可维护性。解析、界面、持久化和抽题策略分离后，后续修改某一部分时不必大范围改动其他模块。
- 题目稳定键的设计解决了题号变化后的记录关联问题，比单纯按题号保存错题更可靠。
- PyQt6 界面相比早期 Tkinter 版本更易组织窗口、弹窗和控制状态，也便于后续继续优化交互体验。
- 手动改题功能弥补了自动解析的不确定性，使程序在面对格式复杂的真实题库时仍能继续使用。

#### 4.2 不足与改进方向

项目目前仍有一些明显的改进空间。首先，题库解析仅支持 txt/docx 等简单纯文本方案，在提高稳定性时牺牲了兼容度，对 PDF、复杂 Word 样式和图片题的支持不足。其次，当前抽题策略主要基于错误次数和错误率，尚未引入更完整的遗忘曲线、保底机制或按知识点分组的复习策略。

后续可以从四个方向继续完善：一是支持更多题库输入形式，例如单词表、Markdown、PDF 类型文件等；二是将基于大模型 API 的解析器接入前端，提高对各类文档的解析能力；三是增加统计图、笔记和解析功能，让用户更直观地认知当前复习进度和掌握水平；四是解耦前端和后端代码，借助 JSON 文件作为中介连接前后端，实现前端和后端独立的更新和维护。